

- 1) Design and implement Parallel Breadth First Search based on existing algorithms using OpenMP. Use a Tree or an

undirected graph for BFS .

```
#include <iostream> // For input and output

#include <stdlib.h> // For standard library functions

#include <queue> // For queue data structure used in BFS

#include <omp.h> // For OpenMP parallel programming

using namespace std;

// Node class for binary tree

class node

{

public:

    node *left, *right; // Left and right child pointers

    int data; // Data stored in node

};

// Function to insert node in binary tree level-wise

node *insert(node *root, int data)

{

    // If tree is empty, create root node

    if (!root)

    {

        root = new node; // Create new node

        root->left = NULL; // Initially left child is NULL

        root->right = NULL; // Initially right child is NULL
```

```
root->data = data; // Store data
```

```
return root; // Return root
```

```
}
```

```
queue<node *> q; // Queue for level-wise insertion
```

```
q.push(root); // Start from root node
```

```
// Continue until correct empty position is found
```

```
while (!q.empty())
```

```
{
```

```
node *temp = q.front(); // Get front node
```

```
q.pop(); // Remove it from queue
```

```
// If left child is empty, insert new node there
```

```
if (temp->left == NULL)
```

```
{
```

```
temp->left = new node;
```

```
temp->left->left = NULL;
```

```
temp->left->right = NULL;
```

```
temp->left->data = data;
```

```
return root;
```

```
}
```

```
else
```

```
{
```

```
q.push(temp->left); // If left child exists, push it into queue
```

```
}
```

```
// If right child is empty, insert new node there
```

```
if (temp->right == NULL)
```

```

{
temp->right = new node;
temp->right->left = NULL;
temp->right->right = NULL;

temp->right->data = data;
return root;
}
else
{
q.push(temp->right); // If right child exists, push it into queue
}
}

return root; // Return root
}

// Function to perform BFS traversal
void bfs(node *head)
{
queue<node *> q; // Queue used for BFS
q.push(head); // Start BFS from root node
int qSize; // Stores number of nodes at current level

// Continue until queue becomes empty
while (!q.empty())
{
qSize = q.size(); // Number of nodes present at current level

```

```

// Process current level nodes in parallel

#pragma omp parallel for
for (int i = 0; i < qSize; i++)
{
    node *currNode; // Current node to be processed

    // Critical section because queue is shared by all threads
    #pragma omp critical
    {
        currNode = q.front(); // Take node from front of queue
        q.pop(); // Remove node from queue
        cout << "\t" << currNode->data; // Print current node
    }

    // Critical section because multiple threads may push children
    #pragma omp critical
    {
        // If left child exists, add it to queue
        if (currNode->left)
            q.push(currNode->left);

        // If right child exists, add it to queue
        if (currNode->right)
            q.push(currNode->right);
    }
}
}

```

```

}

// Main function

int main()

{

node *root = NULL; // Initially tree is empty

int data; // Data to insert


char ans; // User choice for continuing insertion


// Take input from user and insert nodes

do

{

cout << "\nEnter data => ";

cin >> data; // Read data


root = insert(root, data); // Insert data into tree

cout << "Do you want to insert one more node? ";

cin >> ans; // Read user choice


} while (ans == 'y' || ans == 'Y');

cout << "\nBFS Traversal:\n";


bfs(root); // Call BFS function

return 0; // End program

}

```